

app package:

AssetManagementApp.app

```
package app;
import dao.AssetManagementService;
import dao.AssetManagementServiceImpl;
import entity.Asset;
import exception.AssetNotFoundException;
import exception.AssetNotMaintainException;
import java.util.Scanner;
public class AssetManagementApp {
    public static void main(String[] args) throws AssetNotMaintainException {
        Scanner sc = new Scanner(System.in);
        AssetManagementService service = new AssetManagementServiceImpl();
        int choice;
        do {
            System.out.println("\n===== Digital Asset Management System =====");
            System.out.println("1. View All Assets");
            System.out.println("2. Add Asset");
            System.out.println("3. Delete Asset");
            System.out.println("4. Allocate Asset");
            System.out.println("5. Perform Maintenance");
            System.out.println("6. Reserve Asset");
            System.out.println("7. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    service.getAllAssets().forEach(System.out::println);
                    break;
                case 2:
                    sc.nextLine(); // Consume the newline left-over
                    System.out.print("Enter Asset Name: ");
                    String name = sc.nextLine();
                    System.out.print("Enter Asset Type: ");
                    String type = sc.nextLine();
                    System.out.print("Enter Purchase Date (yyyy-mm-dd): ");
                    String date = sc.nextLine();
```

```

    Asset asset = new Asset();
    asset.setName(name);
    asset.setType(type);
    asset.setPurchaseDate(date);
    asset.setSerialNumber("AUTO-GEN-" + System.currentTimeMillis());
// or get input from user
    asset.setLocation("Default"); // or get input
    asset.setStatus("Available"); // default status
    asset.setOwnerId(1);
    service.addAsset(asset);
    System.out.println("Asset added successfully.");
    break;
case 3:
    System.out.print("Enter Asset ID to delete: ");
    int deleteId = sc.nextInt();
    try {
        boolean deleted = service.deleteAsset(deleteId);
        if (deleted) {
            System.out.println("Asset deleted successfully.");
        }
    } catch (AssetNotFoundException e) {
        System.out.println(e.getMessage());
    }
    break;
case 4:
    System.out.print("Enter Asset ID to allocate: ");
    int allocateAssetId = sc.nextInt();
    System.out.print("Enter Employee ID: ");
    int emplId = sc.nextInt();
    sc.nextLine();
    System.out.print("Enter Allocation Date (yyyy-mm-dd): ");
    String allocationDate = sc.nextLine();
    try {
        boolean allocated = service.allocateAsset(allocateAssetId, emplId,
allocationDate);
        if (allocated) {
            System.out.println("Asset allocated successfully.");
        }
    } catch (AssetNotFoundException e) {
        System.out.println(e.getMessage());
    }

```

```

    }
    break;
case 5:
    System.out.print("Enter Asset ID for maintenance: ");
    int maintAssetId = sc.nextInt();
    sc.nextLine();
    System.out.print("Enter Maintenance Date (yyyy-mm-dd): ");
    String mDate = sc.nextLine();
    System.out.print("Enter Description: ");
    String desc = sc.nextLine();
    System.out.print("Enter Cost: ");
    double mCost = sc.nextDouble();
    try {
        boolean maintained = service.performMaintenance(maintAssetId,
mDate, desc, mCost);
        if (maintained) {
            System.out.println("Maintenance record added successfully.");
        }
    } catch (AssetNotFoundException e) {
        System.out.println(e.getMessage());
    }
    break;
case 6:
    System.out.print("Enter Asset ID to reserve: ");
    int resAssetId = sc.nextInt();
    System.out.print("Enter Employee ID: ");
    int resEmpId = sc.nextInt();
    sc.nextLine();
    System.out.print("Enter Reservation Date (yyyy-mm-dd): ");
    String resDate = sc.nextLine();
    System.out.print("Enter Start Date (yyyy-mm-dd): ");
    String startDate = sc.nextLine();
    System.out.print("Enter End Date (yyyy-mm-dd): ");
    String endDate = sc.nextLine();
    try {
        boolean reserved = service.reserveAsset(resAssetId, resEmpId,
resDate, startDate, endDate);
        if (reserved) {
            System.out.println("Asset reserved successfully.");
        }
    }

```

```

        } catch (AssetNotFoundException | AssetNotMaintainException e) {
            System.out.println(e.getMessage());
        }
        break;
    case 7:
        System.out.println("Exiting... Thank you!");
        break;
    default:
        System.out.println("Invalid choice. Try again!");
    }
} while (choice != 7);
sc.close();
}
}

```

=====

dao package:

AssetManagementService.java(interface):

```

package dao;
import java.util.List;
import entity.Asset;
import exception.AssetNotFoundException;
import exception.AssetNotMaintainException;
public interface AssetManagementService {
    boolean addAsset(Asset asset);
    boolean updateAsset(Asset asset);
    boolean deleteAsset(int assetId) throws AssetNotFoundException;
    boolean allocateAsset(int assetId, int employeeId, String allocationDate) throws
AssetNotFoundException;
    boolean deallocateAsset(int assetId, int employeeId, String returnDate) throws
AssetNotFoundException;
    boolean performMaintenance(int assetId, String maintenanceDate, String
description, double cost) throws AssetNotFoundException,
AssetNotMaintainException;

```

```

    boolean reserveAsset(int assetId, int employeeId, String reservationDate, String
startDate, String endDate) throws AssetNotFoundException,
AssetNotMaintainException;
    boolean withdrawReservation(int reservationId);
    List<Asset> getAllAssets();
}

```

```

package dao;
import entity.Asset;
import exception.AssetNotFoundException;
import exception.AssetNotMaintainException;
import util.DBConnUtil;
import java.sql.Connection;
import java.sql.Date;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
public class AssetManagementServiceImpl implements AssetManagementService {
    private Connection conn;
    public AssetManagementServiceImpl() {
        try {
            // Get DB Connection using utility
            conn = DBConnUtil.getConnection();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    @Override
    public boolean addAsset(Asset asset) {
        String query = "INSERT INTO assets (asset_id, name, type, serial_number,
purchase_date, location, status, owner_id) VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
        try (PreparedStatement pstmt = conn.prepareStatement(query)) {
            pstmt.setInt(1, asset.getAssetId());
            pstmt.setString(2, asset.getName());

```

```

        pstmt.setString(3, asset.getType());
        pstmt.setString(4, asset.getSerialNumber());
        pstmt.setString(5, asset.getPurchaseDate());
        pstmt.setString(6, asset.getLocation());
        pstmt.setString(7, asset.getStatus());
        pstmt.setInt(8, asset.getOwnerId());
        int rowsInserted = pstmt.executeUpdate();
        return rowsInserted > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

@Override
public boolean updateAsset(Asset asset) {
    String query = "UPDATE assets SET name = ?, type = ?, serial_number = ?,
purchase_date = ?, location = ?, status = ?, owner_id = ? WHERE asset_id = ?";
    try (PreparedStatement pstmt = conn.prepareStatement(query)) {
        pstmt.setString(1, asset.getName());
        pstmt.setString(2, asset.getType());
        pstmt.setString(3, asset.getSerialNumber());
        pstmt.setString(4, asset.getPurchaseDate());
        pstmt.setString(5, asset.getLocation());
        pstmt.setString(6, asset.getStatus());
        pstmt.setInt(7, asset.getOwnerId());
        pstmt.setInt(8, asset.getAssetId());
        int rowsUpdated = pstmt.executeUpdate();
        return rowsUpdated > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

@Override
public boolean deleteAsset(int assetId) throws AssetNotFoundException {
    Connection conn = null;
    PreparedStatement stmt = null;
    try {
        conn = DBConnUtil.getConnection();
        String query = "DELETE FROM assets WHERE asset_id = ?";

```

```

        stmt = conn.prepareStatement(query);
        stmt.setInt(1, assetId);
        int rows = stmt.executeUpdate();
        if (rows == 0) {
            throw new AssetNotFoundException("Asset with ID " + assetId + " not
found.");
        }
        return true;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

@Override
public boolean allocateAsset(int assetId, int employeeId, String allocationDate)
throws AssetNotFoundException {
    Connection conn = null;
    PreparedStatement stmt = null;
    try {
        conn = DBConnUtil.getConnection();
        // Check if asset exists
        String checkQuery = "SELECT * FROM assets WHERE asset_id = ?";
        stmt = conn.prepareStatement(checkQuery);
        stmt.setInt(1, assetId);
        ResultSet rs = stmt.executeQuery();
        if (rs.next()) {
            throw new AssetNotFoundException("Asset with ID " + assetId + " not
found.");
        }
        // Allocate asset
        String query = "INSERT INTO asset_allocations (asset_id, employee_id,
allocation_date) VALUES (?, ?, ?)";
        stmt = conn.prepareStatement(query);
        stmt.setInt(1, assetId);
        stmt.setInt(2, employeeId);
        stmt.setString(3, allocationDate);
        int rows = stmt.executeUpdate();
        return rows > 0;
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

```

        return false;
    }
}

@Override
public boolean deallocateAsset(int assetId, int employeeId, String returnDate) {
    String query = "UPDATE asset_allocations SET return_date = ? WHERE asset_id = ? AND employee_id = ? AND return_date IS NULL";
    try (PreparedStatement pstmt = conn.prepareStatement(query)) {
        pstmt.setDate(1, Date.valueOf(returnDate)); // Format: "yyyy-mm-dd"
        pstmt.setInt(2, assetId);
        pstmt.setInt(3, employeeId);
        int rowsUpdated = pstmt.executeUpdate();
        return rowsUpdated > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

@Override
public boolean performMaintenance(int assetId, String maintenanceDate, String description, double cost) throws AssetNotFoundException, AssetNotMaintainException {
    Connection conn = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;
    try {
        conn = DBConnUtil.getConnection();
        String checkAssetQuery = "SELECT status FROM assets WHERE asset_id = ?";
        stmt = conn.prepareStatement(checkAssetQuery);
        stmt.setInt(1, assetId);
        rs = stmt.executeQuery();
        if (!rs.next()) {
            throw new AssetNotFoundException("Asset with ID " + assetId + " not found.");
        }
        String assetStatus = rs.getString("status");
        if ("Retired".equalsIgnoreCase(assetStatus)) {
            throw new AssetNotMaintainException("Cannot perform maintenance on a retired asset.");
        }
    }
}

```



```

        if ("Damaged".equalsIgnoreCase(assetStatus)) {
            throw new AssetNotMaintainException("Cannot perform maintenance on a
damaged asset.");
        }
        String insertQuery = "INSERT INTO maintenance_records (asset_id,
maintenance_date, description, cost) VALUES (?, ?, ?, ?)";
        stmt = conn.prepareStatement(insertQuery);
        stmt.setInt(1, assetId);
        stmt.setString(2, maintenanceDate);
        stmt.setString(3, description);
        stmt.setDouble(4, cost);
        int rows = stmt.executeUpdate();
        return rows > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    } finally {
        try { if (rs != null) rs.close(); } catch (SQLException e) { e.printStackTrace(); }
        try { if (stmt != null) stmt.close(); } catch (SQLException e) { e.printStackTrace(); }
    }
}

@Override
public boolean reserveAsset(int assetId, int employeeId, String reservationDate,
String startDate, String endDate) throws AssetNotFoundException,
AssetNotMaintainException {
    Connection conn = null;
    PreparedStatement stmt = null;
    try {
        conn = DBConnUtil.getConnection();
        // Check if asset exists
        String checkQuery = "SELECT * FROM assets WHERE asset_id = ?";
        stmt = conn.prepareStatement(checkQuery);
        stmt.setInt(1, assetId);
        ResultSet rs = stmt.executeQuery();
        if (!rs.next()) {
            throw new AssetNotFoundException("Asset with ID " + assetId + " not
found.");
        }
        // Check last maintenance date

```

```

        String lastMaintenanceQuery = "SELECT MAX(maintenance_date) AS last_date
FROM maintenance_records WHERE asset_id = ?";
        stmt = conn.prepareStatement(lastMaintenanceQuery);
        stmt.setInt(1, assetId);
        rs = stmt.executeQuery();
        if (rs.next()) {
            String lastDate = rs.getString("last_date");
            if (lastDate != null) {
                LocalDate lastMaintenance = LocalDate.parse(lastDate);
                if (lastMaintenance.isBefore(LocalDate.now().minusYears(2))) {
                    throw new AssetNotMaintainException("Asset has not been maintained
in the last 2 years.");
                }
            }
        }
        // Reserve the asset
        String insertQuery = "INSERT INTO reservations (asset_id, employee_id,
reservation_date, start_date, end_date, status) VALUES (?, ?, ?, ?, ?, ?)";
        stmt = conn.prepareStatement(insertQuery);
        stmt.setInt(1, assetId);
        stmt.setInt(2, employeeId);
        stmt.setString(3, reservationDate);
        stmt.setString(4, startDate);
        stmt.setString(5, endDate);
        stmt.setString(6, "pending");
        int rows = stmt.executeUpdate();
        return rows > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

@Override
public boolean withdrawReservation(int reservationId) {
    String query = "UPDATE reservations SET status = ? WHERE reservation_id = ?";
    try (PreparedStatement pstmt = conn.prepareStatement(query)) {
        pstmt.setString(1, "canceled");
        pstmt.setInt(2, reservationId);
        int rowsUpdated = pstmt.executeUpdate();
        return rowsUpdated > 0;
    }
}

```

```

    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

@Override
public List<Asset> getAllAssets() {
    List<Asset> assetList = new ArrayList<>();
    String query = "SELECT * FROM assets";
    try (PreparedStatement pstmt = conn.prepareStatement(query);
        ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            Asset asset = new Asset();
            asset.setAssetId(rs.getInt("asset_id"));
            asset.setName(rs.getString("name"));
            asset.setType(rs.getString("type"));
            asset.setSerialNumber(rs.getString("serial_number"));
            asset.setPurchaseDate(rs.getString("purchase_date"));
            asset.setLocation(rs.getString("location"));
            asset.setStatus(rs.getString("status"));
            asset.setOwnerId(rs.getInt("owner_id"));
            assetList.add(asset);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return assetList;
}
}

```

=====

entity package:

```

package entity;

public class Asset {
    private int assetId;
    private String name;
    private String type;
    private String serialNumber;

```

```

private String purchaseDate;
private String location;
private String status;
private int ownerId;
private double cost;
public Asset() {}
public Asset(int assetId, String name, String type, String serialNumber, String
purchaseDate,
        String location, String status, int ownerId) {
    this.assetId = assetId;
    this.name = name;
    this.type = type;
    this.serialNumber = serialNumber;
    this.purchaseDate = purchaseDate;
    this.location = location;
    this.status = status;
    this.ownerId = ownerId;
}
public int getAssetId() {
    return assetId;
}
public void setAssetId(int assetId) {
    this.assetId = assetId;
}
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public String getType() {
    return type;
}
public void setType(String type) {
    this.type = type;
}
public String getSerialNumber() {
    return serialNumber;
}
public void setSerialNumber(String serialNumber) {

```

```

    this.serialNumber = serialNumber;
}
public String getPurchaseDate() {
    return purchaseDate;
}
public void setPurchaseDate(String purchaseDate) {
    this.purchaseDate = purchaseDate;
}
public String getLocation() {
    return location;
}
public void setLocation(String location) {
    this.location = location;
}
public String getStatus() {
    return status;
}
public void setStatus(String status) {
    this.status = status;
}
public int getOwnerId() {
    return ownerId;
}
public void setOwnerId(int ownerId) {
    this.ownerId = ownerId;
}

    public void setCost(double cost) {
this.cost= cost;
    }
    @Override
    public String toString() {
        return "Asset{" +
            "name=" + name + "\" +
            ", type=" + type + "\" +
            ", purchaseDate=" + purchaseDate + "\" +
            ", cost=" + cost +
            ", serialNumber=" + serialNumber + "\" +
            ", location=" + location + "\" +
            ", status=" + status + "\" +
            ", ownerId=" + ownerId +

```

```
        }  
    }  
}
```

```
package entity;  
public class AssetAllocation {  
    private int allocationId;  
    private int assetId;  
    private int employeeId;  
    private String allocationDate;  
    private String returnDate;  
    public AssetAllocation() {}  
    public AssetAllocation(int allocationId, int assetId, int employeeId, String  
allocationDate, String returnDate) {  
        this.allocationId = allocationId;  
        this.assetId = assetId;  
        this.employeeId = employeeId;  
        this.allocationDate = allocationDate;  
        this.returnDate = returnDate;  
    }  
    public int getAllocationId() {  
        return allocationId;  
    }  
    public void setAllocationId(int allocationId) {  
        this.allocationId = allocationId;  
    }  
    public int getAssetId() {  
        return assetId;  
    }  
    public void setAssetId(int assetId) {  
        this.assetId = assetId;  
    }  
    public int getEmployeeId() {  
        return employeeId;  
    }  
    public void setEmployeeId(int employeeId) {  
        this.employeeId = employeeId;  
    }  
}
```

```

    }
    public String getAllocationDate() {
        return allocationDate;
    }
    public void setAllocationDate(String allocationDate) {
        this.allocationDate = allocationDate;
    }
    public String getReturnDate() {
        return returnDate;
    }
    public void setReturnDate(String returnDate) {
        this.returnDate = returnDate;
    }
}

```

```

package entity;
public class Employee {
    private int employeeId;
    private String name;
    private String department;
    private String email;
    private String password;
    public Employee() {}
    public Employee(int employeeId, String name, String department, String email, String
password) {
        this.employeeId = employeeId;
        this.name = name;
        this.department = department;
        this.email = email;
        this.password = password;
    }
    public int getEmployeeId() {
        return employeeId;
    }
    public void setEmployeeId(int employeeId) {
        this.employeeId = employeeId;
    }
    public String getName() {

```

```

        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getDepartment() {
        return department;
    }
    public void setDepartment(String department) {
        this.department = department;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
}

```

```

package entity;
public class MaintenanceRecord {
    private int maintenancelId;
    private int assetId;
    private String maintenanceDate;
    private String description;
    private double cost;
    public MaintenanceRecord() {}
    public MaintenanceRecord(int maintenancelId, int assetId, String maintenanceDate,
String description, double cost) {
        this.maintenancelId = maintenancelId;
        this.assetId = assetId;
    }
}

```



```

        this.maintenanceDate = maintenanceDate;
        this.description = description;
        this.cost = cost;
    }
    public int getMaintenanceId() {
        return maintenancelId;
    }
    public void setMaintenanceId(int maintenancelId) {
        this.maintenanceId = maintenancelId;
    }
    public int getAssetId() {
        return assetId;
    }
    public void setAssetId(int assetId) {
        this.assetId = assetId;
    }
    public String getMaintenanceDate() {
        return maintenanceDate;
    }
    public void setMaintenanceDate(String maintenanceDate) {
        this.maintenanceDate = maintenanceDate;
    }
    public String getDescription() {
        return description;
    }
    public void setDescription(String description) {
        this.description = description;
    }
    public double getCost() {
        return cost;
    }
    public void setCost(double cost) {
        this.cost = cost;
    }
}

```

```

package entity;
public class Reservation {

```

```

private int reservationId;
private int assetId;
private int employeeId;
private String reservationDate;
private String startDate;
private String endDate;
private String status;
public Reservation() {}
public Reservation(int reservationId, int assetId, int employeeId, String
reservationDate,
                String startDate, String endDate, String status) {
    this.reservationId = reservationId;
    this.assetId = assetId;
    this.employeeId = employeeId;
    this.reservationDate = reservationDate;
    this.startDate = startDate;
    this.endDate = endDate;
    this.status = status;
}
public int getReservationId() {
    return reservationId;
}
public void setReservationId(int reservationId) {
    this.reservationId = reservationId;
}
public int getAssetId() {
    return assetId;
}
public void setAssetId(int assetId) {
    this.assetId = assetId;
}
public int getEmployeeId() {
    return employeeId;
}
public void setEmployeeId(int employeeId) {
    this.employeeId = employeeId;
}
public String getReservationDate() {
    return reservationDate;
}

```

```

public void setReservationDate(String reservationDate) {
    this.reservationDate = reservationDate;
}
public String getStartDate() {
    return startDate;
}
public void setStartDate(String startDate) {
    this.startDate = startDate;
}
public String getEndDate() {
    return endDate;
}
public void setEndDate(String endDate) {
    this.endDate = endDate;
}
public String getStatus() {
    return status;
}
public void setStatus(String status) {
    this.status = status;
}
}

```

=====

exception package:

```

package exception;
public class AssetAlreadyAllocatedException extends Exception {
    public AssetAlreadyAllocatedException(String message) {
        super(message);
    }
}

```

```

package exception;
public class AssetNotFoundException extends Exception {

```

```
public AssetNotFoundException(String message) {  
    super(message);  
}  
}
```

```
package exception;  
public class AssetNotMaintainException extends Exception {  
    public AssetNotMaintainException(String message) {  
        super(message);  
    }  
}
```

```
package exception;  
public class DuplicateAssetException extends Exception {  
    public DuplicateAssetException(String message) {  
        super(message);  
    }  
}
```

```
package exception;  
public class EmployeeNotFoundException extends Exception {  
    public EmployeeNotFoundException(String message) {  
        super(message);  
    }  
}
```

```
package exception;  
public class InvalidAssetAllocationException extends Exception {  
    public InvalidAssetAllocationException(String message) {  
        super(message);  
    }  
}
```

```
}
```

```
-----  
  
package exception;  
public class ReservationNotFoundException extends Exception {  
    public ReservationNotFoundException(String message) {  
        super(message);  
    }  
}
```

```
=====
```

test package:

AssetManagementServiceTest.java :

```
package test;  
import static org.junit.jupiter.api.Assertions.*;  
import org.junit.jupiter.api.BeforeEach;  
import org.junit.jupiter.api.Test;  
import dao.AssetManagementServiceImpl;  
import entity.Asset;  
import exception.AssetNotFoundException;  
import exception.AssetNotMaintainException;  
import java.sql.Connection;  
import java.sql.SQLException;  
import java.sql.Statement;  
import java.util.UUID;  
import util.DBConnUtil;  
public class AssetManagementServiceTest {  
    AssetManagementServiceImpl service;  
  
    @BeforeEach  
    void setUp() {  
        service = new AssetManagementServiceImpl();  
  
        try (Connection conn = DBConnUtil.getConnection();  
            Statement stmt = conn.createStatement()) {
```

```

        stmt.executeUpdate("DELETE FROM assets");
        stmt.executeUpdate("DELETE FROM maintenance_records");
        // Insert a maintainable asset for testMaintenance
        Asset maintainableAsset = new Asset(1, "Laptop", "Computer", "LP789012",
"2024-01-15", "Office", "Available", 2);
        service.addAsset(maintainableAsset);
        // Insert a retired asset for testAssetNotMaintainException
        Asset retiredAsset = new Asset(999, "Old Server", "Server", "SRV34567",
"2019-05-20", "Storage", "Retired", 1);
        service.addAsset(retiredAsset);
    } catch (SQLException e) {
        e.printStackTrace();
        fail("Failed to clean and setup database before test: " + e.getMessage());
    }
}

```

```

@Test
void testAddAsset() {
    String uniqueSerialNumber = "KB" + UUID.randomUUID().toString().substring(0,
10);
    Asset asset = new Asset(0, "Keyboard", "Peripheral", uniqueSerialNumber,
"2023-11-01", "Office", "Available", 1);
    assertTrue(service.addAsset(asset));
}

```

```

@Test
void testAssetNotFoundException() {
    assertTrue(AssetNotFoundException.class, () -> {
        service.allocateAsset(9999, 1, "2024-04-01");
    });
}

```

```

@Test
void testMaintenance() throws AssetNotMaintainException,
AssetNotFoundException {

```

```

    assertTrue(service.performMaintenance(1, "2024-03-01", "Routine check", 150.0));
}

```

```

@Test
void testReservation() throws AssetNotFoundException, AssetNotMaintainException
{

```

```

        assertTrue(service.reserveAsset(1, 1, "2024-04-01", "2024-04-05", "2024-04-10"));
    }
    @Test
    void testAssetNotMaintainException() {

        try {

            service.performMaintenance(999, "2024-04-01", "Attempted Maintenance", 50.0);

            fail("AssetNotMaintainException was not thrown for asset ID 999.");
        } catch (AssetNotMaintainException e) {

            assertTrue(true);

        } catch (AssetNotFoundException e) {
            fail("AssetNotFoundException was thrown instead of
AssetNotMaintainException: " + e.getMessage());
        }
    }
}

```

=====

util package:

DatabaseConnectionChecker.java

```

package util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;

public class DatabaseConnectionChecker {

    public static void main(String[] args) {
        Connection connection = null;
    }
}

```

```

try {
    // Load database properties
    Properties props = DBPropertyUtil.loadProperty();

    if (props == null) {
        System.err.println("Failed to load database properties. Check the file path and
content.");
        return;
    }

    String url = props.getProperty("url");
    String username = props.getProperty("username");
    String password = props.getProperty("password");

    if (url == null || username == null) {
        System.err.println("Database URL or username not found in db.properties.");
        return;
    }

    // Attempt to establish a connection
    connection = DriverManager.getConnection(url, username, password);

    if (connection != null) {
        System.out.println("Successfully connected to the database!");
    } else {
        System.err.println("Failed to establish a database connection.");
    }

} catch (SQLException e) {
    System.err.println("Error establishing database connection: " + e.getMessage());
    e.printStackTrace();
} finally {
    // Close the connection
    if (connection != null) {
        try {
            connection.close();
            System.out.println("Database connection closed.");
        } catch (SQLException e) {
            System.err.println("Error closing database connection: " + e.getMessage());
        }
    }
}

```



```
}  
}  
}  
}
```

DBConnUtil.java

```
package util;  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;  
import java.util.Properties;  
  
public class DBConnUtil {  
  
    public static Connection getConnection() throws SQLException {  
        Properties props = DBPropertyUtil.loadProperty();  
        if (props == null) {  
            throw new SQLException("Failed to load database properties.");  
        }  
        String url = props.getProperty("url");  
        String username = props.getProperty("username");  
        String password = props.getProperty("password");  
  
        if (url == null || username == null) {  
            throw new SQLException("Database URL or username not found.");  
        }  
  
        return DriverManager.getConnection(url, username, password);  
    }  
}
```

DBPropertyUtil.java

```
package util;
```

```

import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;

public class DBPropertyUtil {

    public static Properties loadProperty() {
        Properties props = new Properties();
        // Use ClassLoader to get the resource from the classpath
        try (InputStream is =
DBPropertyUtil.class.getClassLoader().getResourceAsStream("util/db.properties")) {
            if (is == null) {
                System.err.println("Failed to find db.properties in the classpath.");
                return null;
            }
            props.load(is);
        } catch (IOException e) {
            System.err.println("Failed to load database properties: " + e.getMessage());
            e.printStackTrace();
        }
        return props;
    }
}

```

properties.db:

```

url=jdbc:mysql://localhost:3306/digitalassetman
username=root
password=
driverClassName=com.mysql.cj.jdbc.Driver

```

=====

Sql

```

DROP DATABASE IF EXISTS digital_asset_db;
CREATE DATABASE digital_asset_db;

```

```
USE digital_asset_db;
```

```
-- Create Employee Table
```

```
CREATE TABLE employee (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    department VARCHAR(50),  
    email VARCHAR(100) UNIQUE NOT NULL  
);
```

```
-- Create Asset Table
```

```
CREATE TABLE assets(  
    asset_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    type VARCHAR(50),  
    serial_number VARCHAR(100) UNIQUE,  
    purchase_date DATE,  
    location VARCHAR(100),  
    status VARCHAR(20) DEFAULT 'Available',  
    owner_id INT  
);
```

```
-- Create MaintenanceRecord Table
```

```
CREATE TABLE maintenance_records(  
    record_id INT PRIMARY KEY AUTO_INCREMENT,  
    asset_id INT,  
    maintenance_date DATE,  
    description TEXT,  
    cost DECIMAL(10, 2),  
    FOREIGN KEY (asset_id) REFERENCES asset(asset_id) ON DELETE CASCADE  
);
```

```
-- Create AssetAllocation Table
```

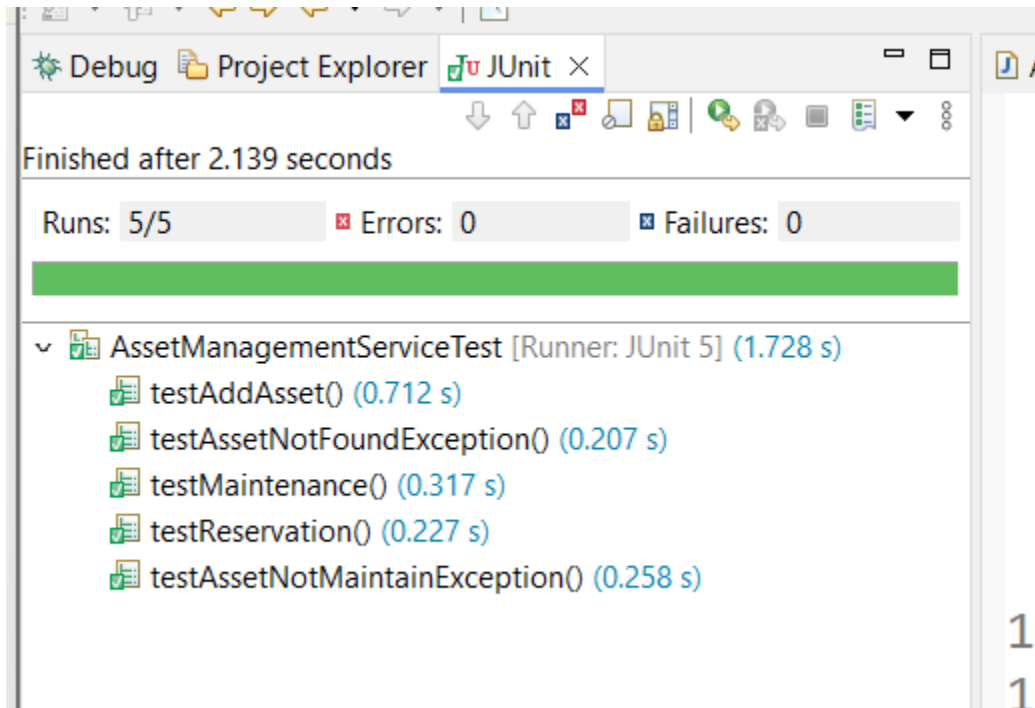
```
CREATE TABLE asset_allocations(  
    allocation_id INT PRIMARY KEY AUTO_INCREMENT,  
    asset_id INT,  
    employee_id INT,  
    allocation_date DATE,  
    return_date DATE,  
    FOREIGN KEY (asset_id) REFERENCES asset(asset_id) ON DELETE CASCADE,
```

```
FOREIGN KEY (employee_id) REFERENCES employee(emp_id) ON DELETE  
CASCADE  
);
```

-- Create Reservation Table

```
CREATE TABLE reservations(  
    reservation_id INT PRIMARY KEY AUTO_INCREMENT,  
    asset_id INT,  
    employee_id INT,  
    reservation_date DATE,  
    start_date DATE,  
    end_date DATE,  
    status VARCHAR(20)  
    FOREIGN KEY (asset_id) REFERENCES asset(asset_id) ON DELETE CASCADE,  
    FOREIGN KEY (employee_id) REFERENCES employee(emp_id) ON DELETE  
CASCADE  
);
```

Test cases:



Outputs:

```
===== Digital Asset Management System =====
1. View All Assets
2. Add Asset
3. Delete Asset
4. Allocate Asset
5. Perform Maintenance
6. Reserve Asset
7. Exit
Enter your choice: 1
Asset{name='Laptop', type='Computer', purchaseDate='2024-01-15', cost=0.0, serialNumber='LP789012', location='Office',
Asset{name='Old Server', type='Server', purchaseDate='2019-05-20', cost=0.0, serialNumber='SRV34567', location='Storage'}
```

```
===== Digital Asset Management System =====
1. View All Assets
2. Add Asset
3. Delete Asset
4. Allocate Asset
5. Perform Maintenance
6. Reserve Asset
7. Exit
Enter your choice: 2
Enter Asset Name: Phone
Enter Asset Type: Small device
Enter Purchase Date (yyyy-mm-dd): 2003-10-12
Asset added successfully.
```

```
===== Digital Asset Management System =====
1. View All Assets
2. Add Asset
3. Delete Asset
4. Allocate Asset
5. Perform Maintenance
6. Reserve Asset
7. Exit
Enter your choice: 1
Asset{name='Laptop', type='Computer', purchaseDate='2024-01-15', cost=0.0, serialNumber='LP789012', location='Office',
Asset{name='Old Server', type='Server', purchaseDate='2019-05-20', cost=0.0, serialNumber='SRV34567', location='Storage',
Asset{name='Phone', type='Small device', purchaseDate='2003-10-12', cost=0.0, serialNumber='AUTO-GEN-1744341943700', lc
```

```
===== Digital Asset Management System =====
```

1. View All Assets
2. Add Asset
3. Delete Asset
4. Allocate Asset
5. Perform Maintenance
6. Reserve Asset
7. Exit

Enter your choice: 3

Enter Asset ID to delete: 1

Asset deleted successfully.

```
s===== Digital Asset Management System =====
```

1. View All Assets
2. Add Asset
3. Delete Asset
4. Allocate Asset
5. Perform Maintenance
6. Reserve Asset
7. Exit

Enter your choice: 4

Enter Asset ID to allocate: 999

Enter Employee ID: 3

Enter Allocation Date (yyyy-mm-dd): 2003-11-12

Asset allocated successfully.

===== Digital Asset Management System =====

1. View All Assets
2. Add Asset
3. Delete Asset
4. Allocate Asset
5. Perform Maintenance
6. Reserve Asset
7. Exit

Enter your choice: 5

Enter Asset ID for maintenance: 1009

Enter Maintenance Date (yyyy-mm-dd): 2003-10-12

Enter Description: Not working

Enter Cost: 2000

Maintenance record added successfully.

===== Digital Asset Management System =====

1. View All Assets
2. Add Asset
3. Delete Asset
4. Allocate Asset
5. Perform Maintenance
6. Reserve Asset
7. Exit

Enter your choice: 6

Enter Asset ID to reserve: 1009

Enter Employee ID: 3

Enter Reservation Date (yyyy-mm-dd): 2003-10-12

Enter Start Date (yyyy-mm-dd): 2004-10-12

Enter End Date (yyyy-mm-dd): 2006-10-12

Asset has not been maintained in the last 2 years.

===== Digital Asset Management System =====

1. View All Assets
2. Add Asset
3. Delete Asset
4. Allocate Asset
5. Perform Maintenance
6. Reserve Asset
7. Exit

Enter your choice: 7

Exiting... Thank you!

===== Digital Asset Management System =====

1. View All Assets
2. Add Asset
3. Delete Asset
4. Allocate Asset
5. Perform Maintenance
5. Reserve Asset
7. Exit

Enter your choice: 2

Enter Asset Name: charger

Enter Asset Type: electronic

Enter Purchase Date (yyyy-mm-dd): 2003-10-13

Asset added successfully.

===== Digital Asset Management System =====

1. View All Assets
2. Add Asset
3. Delete Asset
4. Allocate Asset
5. Perform Maintenance
6. Reserve Asset
7. Exit

Enter your choice: 6

Enter Asset ID to reserve: 1010

Enter Employee ID: 2

Enter Reservation Date (yyyy-mm-dd): 2007-10-12

Enter Start Date (yyyy-mm-dd): 2008-12-14

Enter End Date (yyyy-mm-dd): 2009-10-13

Asset reserved successfully.